PLASTIC-STABLE COMPLEMENTARY LEARNING SYSTEM FOR CONTINUAL LEARNING

Quan Nguyen-Tri[1,2], Anh T. V. Dau[1,2], Le Nam Khanh[2], Linh Ngo Van[2], Hoang Thanh-Tung[3], Khoat Than[2],
[1]FPT Software AI Center, [2]Hanoi University of Science and Technology
[3]University of Engineering and Technology, Vietnam National University Hanoi,

ABSTRACT

The challenge of continual learning (CL) involves addressing two primary issues: the trade-off between plasticity and stability and the inter-task class separation. Recent advances in architecture-based methods show promise in overcoming these challenges by training distinct subnetworks for each task, incorporating Out-of-Distribution (OOD) detection. Nevertheless, these methods encounter limitations related to scalability with constrained memory, the inefficiency of adapting OOD detection in CL due to magnitude imbalance among sub-networks. In our paper, we present the Plasticity-Stability Complementary Learning System (PS-CLS), a novel architecture-based method inspired by the short-term and long-term memory mechanisms of the human brain. PS-CLS effectively balances plasticity and stability within memory constraints. Additionally, we introduce a method for balancing sub-network variance across tasks and propose data augmentation during testing to address inter-task class separation. Our experiments across diverse benchmarks consistently demonstrate that PS-CLS outperforms state-of-the-art methods and is more memory-efficient.

1 Introduction

The challenge posed by Continual Learning (CL) requires the gradual acquisition of a sequence of tasks. The primary hurdle in CL is referred to as catastrophic forgetting [1, 2], a phenomenon wherein the model's performance significantly declines in tasks it had previously learned when confronted with new ones, mainly due to changes in the optimal weights for these tasks. Mitigating catastrophic forgetting involves the agent finding a balance between plasticity (the capacity to learn new tasks quickly but prone to forgetting old ones) and stability (resilience in retaining knowledge of old tasks but slower in acquiring new ones) [3]. In the domain of Continual Learning, two prominent scenarios draw significant attention: Task Incremental Learning (TIL) and Class Incremental Learning (CIL) [4]. The distinguishing factor between TIL and CIL lies in how they handle task identification during evaluation. In TIL, the task identifier is available for predicting the data labels at the evaluation time, whereas in CIL, this task identifier remains inaccessible. CIL introduces a more complex challenge since it involves an additional problem: inter-task class separation.

Various approaches have emerged to address the challenge of continual learning under limited memory constraints. These approaches include memory-based methods [5, 6, 7, 8, 9, 10], which involve the storage of additional information related to the previous tasks, such as data samples and gradients; regularization-based methods [11, 12, 13, 14, 15], which penalize significant parameter changes; and architecture-based methods [16, 17, 18], which entail the learning of distinct sub-networks for each task. Remarkably, architecture-based techniques have proven particularly effective, enabling the development of efficient models for individual tasks while mitigating the challenge of catastrophic forgetting. Furthermore, these techniques can effectively address the issue of inter-task class separation with the assistance of out-of-distribution (OOD) detection mechanisms [18].

Architecture-based methods face a major limitation in their effectiveness when extended to a large number of tasks within limited memory resources. In order to maintain sufficient capacity for remembering all tasks, [19, 20, 21] require a rapid expansion of the network architecture, while [22, 17] offer a more scalable approach for handling a large number of tasks, which comes at the expense of task-specific performance. Moreover, addressing the inter-task class separation problem effectively necessitates efficient utilization of limited buffer memory to store data from previous tasks. An impactful approach involves augmenting the OOD detection capability of subnetworks [18] through the use

of specialized training and evaluation techniques [23, 24]. Nevertheless, the simple combination of OOD detection with architecture-based methods may face challenges related to the magnitude of the imbalance of outputs from different tasks. This is attributed to the individual training of each task in such methods [18].

In this paper, we present Plasticity-Stability Complementary Learning System (PS-CLS), a novel architecture-based method designed to address the above-mentioned issues. We introduce a novel architecture that inspired by the learning mechanisms of the human brain's long-term and short-term memory to tackle the trade-off between memory and performance. This system exploits analogies between short-term memory and plasticity, as well as long-term memory and stability. The plastic network is specifically tailored for capturing plasticity, rapidly acquiring new knowledge, and exhibiting quick forgetfulness akin to short-term memory through an iterative expansion and compression process. Conversely, the stable network is designed to embody stability, preserving performance on each task by storing knowledge learned from the plastic network and gradually transferring it to long-term memory. The stable network efficiently utilizes memory by learning a binary mask from the weights obtained from the plastic network. Using a gradual knowledge distillation mechanism, we simulate the transfer process between long-term and short-term memories. Furthermore, to address the inter-task class separation problem, we propose constraining the output variance to mitigate the magnitude imbalance among task-specific sub-networks. Data augmentations are then utilized during training and testing to enhance Out-Of-Distribution detection capabilities and improve the estimation of the In-Distribution data distribution. Extensive experiments demonstrate that our method achieves new state-of-the-art performance in both Task Incremental Learning (TIL) and Class Incremental Learning (CIL) scenarios, while also utilizing memory more efficiently.

2 Related works

Research on addressing continual learning has undergone a lengthy development process and is broadly categorized into three main approaches: regularization-based [11, 12, 13, 14, 15, 16, 25, 26, 27], memory-based [5, 6, 7, 8, 9, 10], and architecture-based approaches [16, 17, 18, 22]. The family of architecture-based methods has proven to be highly effective, employing parameter isolation to mitigate the issue of catastrophic forgetting. Additionally, OOD detection methods [24] are often combined with architecture-based approaches to tackle the challenges in CIL [18]. The ensemble technique [22, 28] is also integrated with a cost-effective architecture-based method to mitigate the issue of overconfidence when predicting for CIL.

Moreover, the fusion of architecture-based and memory-based approaches [29, 30, 31, 32] has garnered attention due to its novelty and its connections to the human biological brain. Specifically, dual-network architectures [29, 32] designed to emulate the "hippocampus" and "neocortex" components of the brain present a promising direction for constructing a network architecture that closely mimics the workings of the human brain. Our proposed method is also designed to simulate the Complementary Learning System (CLS) [2, 33] in the human brain. However, instead of employing a universal CLS architecture for all tasks, we incorporate CLS with parameter isolation. Moreover, our method integrates OOD detection and ensemble techniques to establish an effective continual learning system.

3 Method

In this section, we outline our approach for balancing plasticity and stability as well as addressing inter-task class separation in Class Incremental Learning. The overview of our proposed method is illustrated in Figure 1. First, we introduce our novel architecture inspired by the Complementary Learning System in Section 3.1. Then, we explain how we mitigate the imbalance between sub-networks in Section 3.2. Finally, in Section 3.3, we detail our approach to tackling inter-task class separation using the proposed architecture.

Notation: In CL we sequentially learn a series of tasks denoted by $1, 2, \ldots, T$. Each task t comprises a training dataset $\mathcal{D}^t = \{(\mathbf{x}^t, \mathbf{y}^t)\}_{i=1}^{n^t}$, where n^t represents the number of data samples in task t. Here, $\mathbf{x}^t$ denotes an input sample, and $\mathbf{y}^t$ denotes its corresponding class label. During the training of task t, the model has access only to the training data of that task $\mathcal{D}^t$ and a small buffer $\mathcal{M}^t$, which stores samples from previous tasks. The distinction between CIL and TIL lies in their inference processes. TIL necessitates task identity during inference, whereas CIL does not.

3.1 Plasticity-Stability Complementary Learning System

Drawing inspiration from the Complementary Learning System (CLS), we propose to train both a Plastic network and a Stable network for each task. The Plastic network serves as the short-term memory, emphasizing rapid acquisition of pattern-separated representations for specific experiences while also gradually forgetting due to capacity restriction. In contrast, the Stable network functions as the long-term memory, gradually assimilating knowledge transferred from

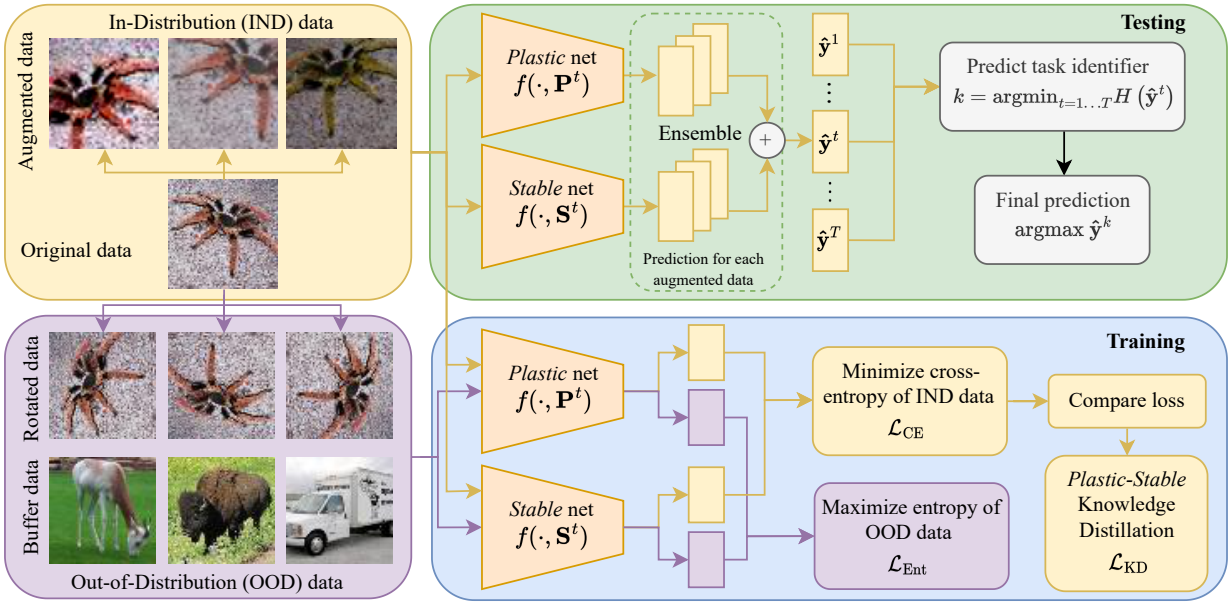
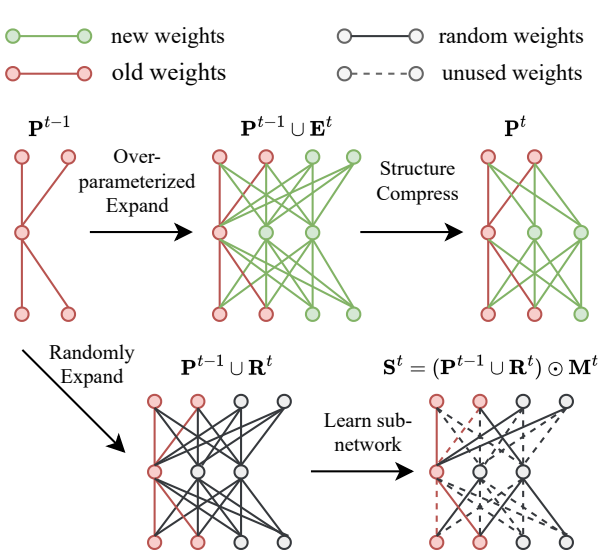
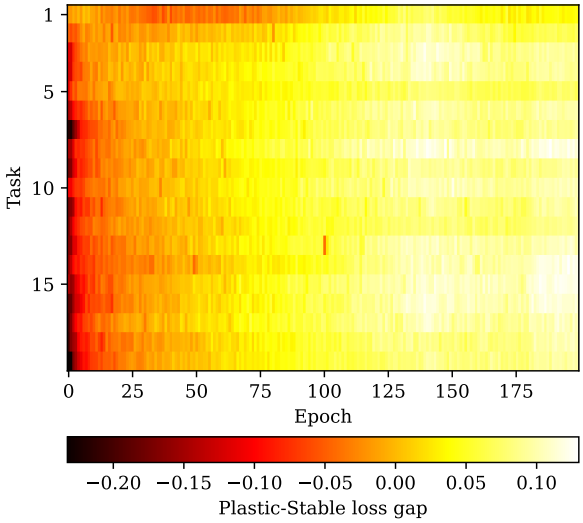


Figure 1: **An overview of our proposed method, PS-CLS**. For each task, we train both a Plastic and a Stable network to minimize cross-entropy loss on augmented IND data. Additionally, during training, the Plastic and Stable networks complement each other via gradual knowledge distillation. To address inter-task class separation, we maximize the entropy of model predictions associated with buffer data and pseudo OOD data generated by rotation. During inference, we employ ensemble prediction from both Plastic and Stable networks. The final prediction is determined by selecting the most confident sub-network prediction across all learned tasks. Furthermore, data augmentation is utilized to improve the estimation of the test sample distribution during inference.



(a) The structure of plastic and stable networks.



(b) The loss gap between Plastic and Stable networks during the training of each task.

Figure 2: **Plastic-Stable complementary learning system**. (2a): Given the Plastic weights $\mathbf{P}^{t-1}$ for the previous task, the Plastic weights $\mathbf{P}^t$ for a new task are formed by expanding them with new learnable weights and then compressing them after learning. The Stable weights $\mathbf{S}_l^t$ for a new task are created by learning a binary mask on top of $\mathbf{P}^{t-1}$ combined with random weights $\mathbf{R}^t$ generated from a fixed random seed. Only the Plastic weights $\mathbf{P}^t$ and the binary mask $\mathbf{M}^t$ are retained. (2b): The smaller loss gap (Plastic loss minus Stable loss) observed in the earlier training stages indicates that the Plastic network performs better initially, while the Stable network gradually performs better at the latter training stage. The Plastic network benefits from knowledge transfer, resulting in a wider loss gap during the earlier training stages for subsequent tasks.

the short-term memory for long-term retention. Figure 2a illustrates our architecture design for the Plastic and Stable networks, while Figure 2b depicts the behavior of the Plastic network as short-term memory and the Stable network as long-term memory.

Plastic network: Learning the Plastic network involves two steps: *over-parameterized expansion* and *structural compression*. In the *over-parameterized expansion* phase, we augment the Plastic weights for previous task with new learnable weights. Formally, the weights corresponding to the sub-network at the l^{th} layer of the t^{th} task can be expressed as: $\mathbf{P}_l^t = \mathbf{P}_l^{t-1} \cup \mathbf{E}_l^t$. Here, $\mathbf{E}_l^t$ represents the new weights added to learn the t^{th} task, and $\mathbf{P}_l^0 = \emptyset$. $\mathbf{P}_l^{t-1}$ is frozen to prevent forgetting. We ensure that the number of new learnable weights for each task, denoted as $|\mathbf{E}_l^t|$, is equal to the number of weights in the backbone architecture (e.g., ResNet-18 [34]). This approach maintains the over-parameterized property of the Plastic network, enabling it to converge to global minima even when using a simple algorithm such as stochastic gradient descent (SGD) [35]. During the *structure compression* phase, we remove unimportant neurons, resulting in a smaller, denser network that directly reduces memory and computation costs. We adopt a modified version of Group Lasso regularization for learning the number of neurons [36] in this phase. This approach will be introduced in Section 3.2, along with our inter-task variance balancing sub-networks.

Stable network: We regard the Plastic weights $\mathbf{P}_l^{t-1}$ learned from the previous task as a Knowledge Base. The Stable network is trained by identifying a binary mask $\mathbf{M}_l^t$ to adjust the weights from the Knowledge Base to suit the new task. Firstly, we pad the Knowledge Base with random weights $\mathbf{R}_l^t$ generated from a fixed random seed, representing empty knowledge. Then, we obtain the binary mask $\mathbf{M}_l^t$ on top of $\mathbf{P}_l^{t-1} \cup \mathbf{R}_l^t$ by optimizing learnable parameters using the edge-popup algorithm [37] (please refer to the supplementary materials for further details). Finally, the weights for the Stable network are constructed as follows: $\mathbf{S}_l^t = (\mathbf{P}_l^{t-1} \cup \mathbf{R}_l^t) \odot \mathbf{M}_l^t$. The total number of weights in $\mathbf{P}_l^{t-1} \cup \mathbf{R}_l^t$ is adjusted to be greater than or equal to the number of weights in the backbone architecture to leverage the benefits of the over-parameterized property. Random weights are generated from a fixed seed, eliminating the need to store them [17]. Consequently, only a binary mask $\mathbf{M}_l^t$ needs to be stored for each task, which can be efficiently encoded using 7-bit Huffman encoding [38].

When training the t^{th} task, we optimize task-specific Plastic and Stable networks with the cross-entropy objective on the training data $\mathcal{D}^t$:

$$\mathcal{L}_{\text{CE}} = \sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t} \big(L(f(\mathbf{x}^t, \mathbf{P}^t), \mathbf{y}^t) + L(f(\mathbf{x}^t, \mathbf{S}^t), \mathbf{y}^t) \big) \tag{1}$$

where $L(\cdot, \cdot)$ denotes the cross-entropy loss, $f(\cdot, \cdot)$ denotes the backbone architecture (e.g., ResNet18 [34]).

Plastic-Stable Gradual Knowledge Distillation: The *over-parameterized expansion* and knowledge transfer from reusing old weights provide significant benefits to the Plastic network in the earlier training stages (see Figure 2b), but this advantage diminishes quickly during the *structure compression* phase. This behavior makes the Plastic network akin to short-term memory. On the other hand, the Stable network adjusts the learned weights to suit the new task, leading to slower learning but greater efficiency in long-term memory storage. During the training of each task, we gradually transfer knowledge from short-term memory to long-term memory through Gradual Knowledge Distillation. Specifically, we compare the cross-entropy loss of the Plastic and Stable networks for each data point to determine which network performs better. The superior network then serves as a teacher to distill knowledge to the other network. The objective of gradual knowledge distillation can be expressed as follows:

$$\begin{aligned} \mathcal{L}_{\text{GKD}} = \sum_{(\mathbf{x}^t, \mathbf{y}^t) \in \mathcal{D}^t} \big(& b(\mathbf{x}^t, \mathbf{y}^t) D_{\text{KL}}(f(\mathbf{x}^t, \mathbf{P}^t) \| f(\mathbf{x}^t, \mathbf{S}^t)) \\ & + (1 - b(\mathbf{x}^t, \mathbf{y}^t)) D_{\text{KL}}(f(\mathbf{x}^t, \mathbf{S}^t) \| f(\mathbf{x}^t, \mathbf{P}^t)) \big) \end{aligned} \tag{2}$$

where $b(\mathbf{x}^t, \mathbf{y}^t) = \mathbb{1}_{L(f(\mathbf{x}^t, \mathbf{P}^t), \mathbf{y}^t) > L(f(\mathbf{x}^t, \mathbf{S}^t), \mathbf{y}^t)}$. This objective facilitates the gradual consolidation of knowledge learned by the Plastic network during earlier training into the Stable network, as well as the reciprocal transfer from the Stable network back to the Plastic network. This process prevents the loss of knowledge acquired by the Plastic network, enabling effective utilization of both networks during inference.

3.2 Inter-task variance balancing

The variance resulting from training multiple sub-networks for each task hampers the model's capability to differentiate classes across different tasks and diminishes the efficacy of reusing weights from previous tasks for knowledge transfer. To address this problem, we propose constraining the weight distribution during training, thereby reducing the variance introduced throughout the training process in the final sub-network.

Variance Constrain: We assume that the weights in each task follow a different distribution; the weights of each neuron follow the same distribution, denoted by $w_{l,n}^t$. Following [39] we can estimate the output variance of the neurons corresponding to the t^{th} task in the l^{th} layer when $\mathbb{E}[w_{l,n}^t] = 0$, as follows:

$$\text{Var}[x_{l,n}^t] = K_l \text{Var}[w_{l,n}^t] \sum_{m=1}^{N_{l-1}^t} \frac{\text{Var}[x_{l-1,m}^t]}{\alpha_{l-1}} \tag{3}$$

where K_l is the kernel size of the l^{th} layer, and α_l is the constant that depends on the activation function at the l^{th} layer. The detailed derivation is provided in the supplementary materials. Hence, the sum output variance of each layer changes exponentially depending on the variance of the weights, as follows:

$$\sum_{n=1}^{N_l^t} \frac{\text{Var}[x_{l,n}^t]}{\alpha_l} = \frac{K_l}{\alpha_l} \sum_{n=1}^{N_l^t} \text{Var}[w_{l,n}^t] \sum_{m=1}^{N_{l-1}^t} \frac{\text{Var}[x_{l-1,m}^t]}{\alpha_{l-1}} \tag{4}$$

To prevent the exponential reduction or amplification of output signal magnitudes and to maintain a balance in the magnitudes of output signals across tasks, our objective is to constrain the variance of the weights as:

$$\frac{K_l}{\alpha_l} \sum_{n=1}^{N_l^t} \text{Var}[w_{l,n}^t] = 1, \quad \mathbb{E}[w_{l,n}^t] = 0, \quad \forall l = \{1, \ldots, L\} \tag{5}$$

We also present adaptive versions of this constraint, incorporating popular techniques in deep learning such as residual connections and batch normalization, in the supplementary materials. By applying constraint (5), we can derive the output variance of the final layer as: $\sum_{n=N_L^{t-1}}^{N_L^t} \text{Var}[x_{L,n}^t] = \sum_{m=1}^{N_0} (\text{Var}[x_{0,m}^t] + (\mathbb{E}[x_{0,m}^t])^2)$ .Here, N_0 denotes the number of channels of the input images, and $\mathbf{x_0^t}$ represents the input of the t^{th} task. It is worth noting that the output of the last layer is task-specific, and the number of input features is not expanded. Meeting condition (5) not only prevents the exponential decrease or increase in output signal magnitudes but also improves the estimation of the output distribution for each task, mitigating the adverse effects of variance during training.

Weight Normalization (WN): To constrain the variance of the weights during training, we propose a simple weight normalization and initialization method. Namely, we initialize the weights for the new task using the same distribution: $w_{l,n}^t \sim \mathcal{N}(0, \frac{\alpha_l}{K_l N_l^t})$. While training, we normalize the weight distribution in the following manner: $\mathbf{P}_{l,n,:}^t = \frac{\sqrt{\alpha_l}(\mathbf{P}_{l,n,:}^t - \mathbb{E}[w_{l,n}^t])}{\sqrt{K_l \sum_{m=1}^{N_l} \text{Var}[w_{l,m}^t]}}$ where the mean and variance of weights associated with each neuron are computed after each step of stochastic gradient descent, as follows: $\mathbb{E}[w_{l,n}^t] = \frac{1}{K_l N_{l-1}^t} \sum_{i=1}^{K_l N_{l-1}^t} \mathbf{P}_{l,n,i}^t$ and $\text{Var}[w_{l,n}^t] = \frac{1}{K_l N_{l-1}^t} \sum_{i=1}^{K_l N_{l-1}^t} \left(\mathbf{P}_{l,n,i}^t - \mathbb{E}[w_{l,n}^t]\right)^2$.

The subtraction of the mean from the weight distribution behaves similarly to the weight decay term used in regularizing neural network training. However, instead of affecting individual weights, it impacts the mean of the weight distribution. Conversely, the variance term serves as a scaling factor for weights, controlling their magnitude without altering the network's predictions.

Knowledge Integration: In our expansion scenario, the weights of the new task, $\mathbf{P}_l^t$, also encompass the weights of the old tasks, $\mathbf{P}_l^{t-1}$. To seamlessly integrate the old and new weights without altering the former, we straightforwardly disregard the old weights by index when estimating the distribution of the new weights. Additionally, we normalize the distribution of old weights to align with the distribution of the new ones, given our assumption that the weights of each neuron follow the same distribution: $\mathbf{P}_{l,n,:}^{t-1} = \mathbf{P}_{l,n,:}^{t-1} \sqrt{\frac{\text{Var}[w_{l,n}^t]}{\text{Var}[w_{l,n}^{t-1}]}}$ Note that we refrain from renormalizing the mean of the weight distribution, as they are both equal to zero due to weight normalization. Incorporating old weights into the sub-network of the new task in this manner allows knowledge from old tasks to be transferred for improved learning of new tasks without the negative effects of imbalance.

Variance-based pruning: Thanks to the variance measurement of each neuron in equation (3), we can employ the pruning of neurons with small output variance. Since these neurons likely have a constant output, their removal has little effect on performance, especially given that our weight normalization step enforces the mean of their output to become zero. Since the output variance of each neuron only depends on the variance of their weights and the

output variance of the previous layer, we adopt a modified version of Group Lasso regularization for learning the number of neurons in a neural network [36], specifically to quantify the weight variance rather than the weight norm: $\mathcal{R} = \sum_{l=1}^{L-1} \sum_{n=N_l^{t-1}}^{N_l^t} \sqrt{\text{Var}[w_{l,n}^t]}$, We exclusively prune the new neurons $N_l^{t-1}, \ldots, N_l^t$. The regularization can be optimized via proximal gradient descent, similar to the approach in [36]. Please refer to the supplementary materials for further details about the training algorithm.

3.3 Inter-task class separation

Recent study [24] has demonstrated the effectiveness of distributional shift augmentation, particularly rotation, in generating OOD data. Rotation augmentation has also been shown to improve OOD detection in continual learning (CL) [18]. Building on this, we utilize rotation to create examples of OOD data, aiding the model in distinguishing between IND and OOD data. Additionally, we use buffer data $\mathcal{M}^t$ as OOD data. While the sub-networks of each task are optimized to minimize cross-entropy on IND data, we also maximize entropy on the OOD data as follows:

$$\mathcal{L}_{\text{Ent}} = - \sum_{\tilde{\mathbf{x}}^t \in \tilde{\mathcal{D}}^t \cup \mathcal{M}^t} \left(H(f(\tilde{\mathbf{x}}^t, \mathbf{P}^t)) + H(f(\tilde{\mathbf{x}}^t, \mathbf{S}^t)) \right) \tag{6}$$

Here, we rotate the t^{th} task data at a random angle from 90º, 180º, and 270º to create additional OOD data $\tilde{\mathcal{D}}^t$ for the objective (6). During inference, for each data point $\mathbf{x}$, we make predictions using all sub-networks for all learned tasks to obtain task-specific predictions $\hat{\mathbf{y}}^t$. Subsequently, we obtain the final prediction by selecting the most confident ones: $t = \text{argmin}_{t=1\ldots T} H\left(\hat{\mathbf{y}}^t\right)$, where $H(\mathbf{x}) = \sum_i x_i \log(x_i)$ denotes the entropy. The inter-task variance balancing method introduced in Section 3.2 helps mitigate variance during training, strengthening the dependency of the output distribution on the input distribution (refer to equation (**??**)). However, estimating the distribution of a single data point during inference poses challenges. One approach to tackle this is by performing predictions on multiple samples at a time, assuming they all have the same distribution. However, this assumption may not hold in real-world scenarios. To address this, we propose using Data Augmentation during Testing (DAT) to better estimate the distribution of a single data point during inference. We leverage the common practice of using data augmentation during training neural networks, arguing that the model is well-trained on the approximated data distribution generated from data augmentation. Therefore, during testing, we use the same data augmentation $\mathcal{T}(\cdot)$ family that was used to train the model to estimate the distribution of a test sample, as follows:

$$\hat{\mathbf{y}}^t = \mathbb{E}_{\tilde{x} \sim \mathcal{T}(x)} \left[\frac{1}{2} \left(f(\tilde{x}, \mathbf{P}^t) + f(\tilde{x}, \mathbf{S}^t) \right) \right] \tag{7}$$

Here, we utilize $\mathcal{T}(\cdot)$ to generate multiple augmented versions $\tilde{x}$ and then compute the average prediction of these augmented versions. Additionally, we employ ensemble prediction using both the Plastic and Stable networks.

4 Experiments

We follow the evaluation protocol outlined in [12, 9, 11, 18] and evaluate our method across various benchmarks in both TIL and CIL scenarios. The CIFAR-100 [40] dataset comprises 100 classes, divided into 10 tasks (CIFAR100/10T) and 20 tasks (CIFAR100/20T), with each task containing 5 and 10 classes, respectively. The Tiny ImageNet [41] dataset consists of 200 classes, divided into 5 tasks (TinyImg/5T) and 10 tasks (TinyImg/10T), with each task having 5 and 10 classes. In all scenarios, we utilize ResNet18 [34] as the backbone structure. Unlike previous SOTA [18] that double the number of neurons in the original architecture to accommodate a larger number of classes, we opt for the base version of ResNet18 [34]. This decision is based on our method's capability to dynamically adapt the number of neurons required for learning tasks efficiently. Additional experimental settings and hyper-parameters information can be found in the supplementary materials.

Baselines: We incorporate various families of CL methods, spanning regular-ization-based, memory-based, and architecture-based. MUC [12], and PASS fall under the regularization-based category. Memory-based methods include LwF.R [6], iCaRL [10], Mnemonics [7], BiC [8], DER++ [9], and OWM [5]. Lastly, architecture-based methods include HAT [16], SupSup [17] (Sup), HAT+CSI [18] and Sup+CSI [18].

4.1 Main Results

As shown in the results presented in Table 1, PS-CLS outperforms all baseline methods, achieving a new state-of-the-art performance. Particularly in the CIL scenario, we observe significant improvements over baselines. The margin of

Table 1: **Average accuracy after all tasks are learned**. "TIL" represents the average Task Incremental Learning accuracy, indicating the average performance when tasks are learned sequentially. "CIL" stands for the average Class Incremental Learning accuracy, reflecting the average performance when classes are learned incrementally. "NP" denotes the number of parameters in millions used at inference after learning the final task. The names of rehearsal-free methods are italicized. We employ a fixed-size memory buffer containing 2000 samples for rehearsal-based methods across all scenarios.

Method	CIFAR100/10T			CIFAR100/20T			TinyImg/5T			TinyImg/10T		
	TIL(%)↑	CIL(%)↑	NP↓	TIL(%)↑	CIL(%)↑	NP↓	TIL(%)↑	CIL(%)↑	NP↓	TIL(%)↑	CIL(%)↑	NP↓
OWM [5]	59.9±0.84	29.0±0.72	05.36	65.4±0.07	24.2±0.11	05.36	22.4±0.87	10.0±0.55	05.46	28.1±0.55	08.6±0.42	05.46
MUC [12]	76.9±1.27	30.0±1.37	45.06	73.7±1.27	14.4±0.93	45.06	55.8±0.26	33.6±0.18	45.47	47.2±0.22	17.4±0.17	45.47
PASS [11]	72.4±1.23	36.8±1.64	44.76	76.9±0.77	25.3±0.81	44.76	50.3±1.97	28.9±1.36	44.86	47.6±0.38	18.7±0.58	44.86
HAT [16]	84.0±0.23	41.1±0.93	45.01	85.0±0.85	26.0±0.83	45.28	61.2±0.72	38.5±1.85	44.97	63.8±0.41	29.8±0.65	45.11
Sup [17]	85.2±0.25	33.1±0.47	05.75	88.8±0.18	12.3±0.30	11.45	61.0±0.62	36.9±0.57	02.95	64.4±0.20	27.0±0.45	05.80
HAT+CSI [18]	92.0±0.37	63.3±1.00	45.31	94.3±0.06	54.6±0.92	45.58	68.4±0.16	45.7±0.26	45.59	72.4±0.21	47.1±0.18	45.72
Sup+CSI	93.0±0.13	66.6±0.23	05.90	95.3±0.20	60.5±0.89	11.60	65.9±0.25	47.7±0.30	03.04	74.1±0.28	46.3±0.30	06.05
LWF.R [6]	86.2±1.00	45.3±0.75	44.76	89.0±0.45	44.3±0.46	44.76	56.4±0.48	32.2±0.50	44.86	55.3±0.35	24.3±0.26	44.86
iCaRL [10]	84.2±1.04	51.4±0.99	44.76	85.7±0.68	47.8±0.48	44.76	54.3±0.59	37.0±0.41	44.86	52.7±0.37	28.3±0.18	44.86
Mnemonics [7]	82.3±0.30	51.0±0.34	44.76	86.2±0.46	47.6±0.74	44.76	54.8±0.16	37.1±0.46	44.86	52.9±0.66	28.5±0.72	44.86
BiC [8]	87.6±0.28	51.3±0.59	44.76	90.3±0.26	40.1±0.77	44.76	44.7±0.71	20.2±0.31	44.86	50.3±0.65	21.2±0.46	44.86
DER++ [9]	84.2±0.47	55.3±0.10	44.76	86.6±0.50	46.6±1.44	44.76	58.0±0.52	36.0±0.42	44.86	59.7±0.60	30.5±0.30	44.86
HAT+CSI [18]	92.0±0.37	65.2±0.71	45.31	94.3±0.06	58.0±0.45	45.58	68.4±0.16	51.7±0.37	45.59	72.4±0.21	47.6±0.32	45.72
Sup+CSI [18]	93.0±0.13	67.0±0.14	05.90	95.3±0.20	60.4±1.04	11.60	65.9±0.25	48.2±0.35	03.04	74.1±0.28	46.1±0.32	06.05
PS-CLS (ours)	93.0±0.06	71.1±0.17	09.67	95.4±0.09	67.5±0.15	17.34	73.2±0.18	53.5±0.10	14.69	79.0±0.12	51.4±0.10	19.27

Table 2: Ablation study on each component of our proposed method on CIFAR100/20T and TinyImg/10T. Plastic or Stable refers to when evaluating using only Plastic or Stable network, "w/o GKD" indicates the exclusion of gradual knowledge distillation in the loss function (2). "w/o VB" implies the absence of the inter-task variance balancing technique introduced in Section 3.2. "w/o DAT" denotes the omission of data augmentation during testing , refer to equation (7).

Ablation	CIL(%)↑						TIL(%)↑					
	CIFAR100/20T			TinyImg/10T			CIFAR100/20T			TinyImg/10T		
	Plastic	Stable	Both	Plastic	Stable	Both	Plastic	Stable	Both	Plastic	Stable	Both
Base	63.2	67.3	67.2	49.2	50.1	51.4	94.9	95.4	95.4	77.5	78.2	78.8
w/o GKD	58.1	63.9	63.8	49.4	48.9	50.7	93.2	94.8	94.7	77.0	77.3	78.7
w/o VB	63.3	64.4	65.0	47.7	48.4	49.3	94.8	94.7	95.1	76.6	77.3	78.0
w/o DAT	56.7	59.3	60.8	42.3	43.2	45.2	91.9	91.3	92.9	72.4	73.0	74.6

improvement ranges from a minimum of 4.1% (higher than Sup+CSI on CIFAR-100 with 10 tasks) to a maximum of 7.0% (higher than Sup+CSI on CIFAR-100 with 20 tasks). In other words, the proposed method consistently outperforms the baselines by a significant margin across different scenarios and task complexities. In the two CIFAR-100 scenarios, our method attains the same TIL performance as Sup+CSI [18], yet the notably higher CIL results suggest that our approach effectively tackles the challenge of inter-task class separation. This highlights the superiority of our proposed method over directly combining Out-of-Distribution (OOD) detection with a robust architecture-based method (Sup+CSI [18], HAT+CSI [18]). In the two scenarios on Tiny ImageNet, our method achieves approximately 5% higher Task Incremental Learning (TIL) performance than the second-ranked baseline. This can be attributed to the fact that in scenarios on Tiny ImageNet, each task's data encompasses more classes (20-40 classes per task and 200 classes in total). This directly challenges the ability of architecture-based methods (e.g., HAT [16], Sup [17]) to balance plasticity and stability with limited memory. HAT [16], despite using three times as many parameters as our method, still appears inferior. Meanwhile, Sup [17], although using the least amount of memory, reveals its disadvantages when the tasks are highly complex, accentuating the performance gap between weight learning and mask learning [37]. In summary, our method outperforms the baselines significantly in both scenarios, where there is a large number of tasks and high complexity for each task. Importantly, it achieves this superior performance while requiring much less memory than the baselines. These results confirm the effectiveness of the proposed method in addressing the challenges outlined in Continual Learning.

4.2 Ablation Study

Experiments for the ablation study are conducted, and the results are presented in Table 2. As a whole, each proposed component significantly contributes to the overall performance of PS-CLS. While the Stable network demonstrates greater efficiency resembling long-term memory behavior, combining predictions from both Plastic and Stable models yields superior results, particularly in challenging tasks like Tiny-ImageNet.

Method	w/o DAT	w/ DAT
Sup+CSI (SupCLR)	59.3	63.2
Sup+CSI (Cross-Entropy)	53.2	49.5
PS-CLS (Cross-Entropy)	60.8	67.2

(a) Using DAT on baseline (Sup+CSI).

Dataset	None	Rotation	Buffer	Both
CIFAR100/20T	48.0	58.5	60.8	67.2
TinyImg/10T	45.7	49.1	47.3	51.4

(b) The effectiveness of OOD data

Table 3: **Left**: The CIL (%) performance on CIFAR100/20T when applying our proposed DAT to the Sup+CSI baseline method. Here, "Sup+CSI (SupCLR)" refers to the original approach using Supervised Contrastive loss (SupCLR) from [18], while "Sup+CSI (Cross-Entropy)" denotes our re-implementation using Cross-Entropy loss, consistent with our PS-CLS method. **Right**: The impact of OOD data on the CIL (%) performance. Here, "None" indicates not using any OOD data to optimize (6), while "Rotation" and "Buffer" signify the utilization of rotated data and buffer data, respectively, as OOD data.

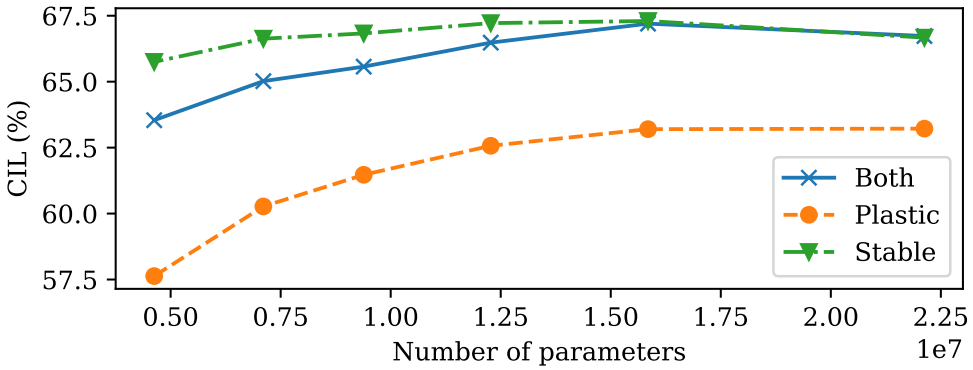


Figure 3: Memory and performance trade-off on CIFAR-100/20T.

w/o GKD: The results exhibit a notable decline in performance when gradual knowledge distillation is omitted. This gradual transfer of temporary knowledge from short-term memory to long-term memory, and vice versa, is crucial in the Complementary Learning System of Plastic and Stable sub-networks. Particularly, short-term memory performance gradually diminishes due to the imposed memory constraints, leading to a gradual loss of the knowledge it acquired. Meanwhile, long-term memory is unable to leverage the knowledge absorbed quickly by short-term memory in the initial steps; it relies solely on the fixed knowledge compressed during previous tasks.

w/o VB: Neglecting the proposed variance balancing method also significantly reduces CIL performance. This decline is primarily attributed to the imbalance in magnitudes among sub-networks, resulting in a weakened ability to detect OOD instances. Conversely, normalizing weights during training aids in better integrating knowledge from old tasks with the knowledge of new tasks through the distributional matching mechanism. This contributes to enhancing the efficiency of Task Incremental Learning (TIL).

w/o DAT: The incorporation of data augmentation during testing stands out as a crucial factor contributing to the superiority of our method over the baselines. Note that HAT+CSI [18] and Sup+CSI [18] are empowered with the Supervised-Contrastive loss (SupCLR), which significantly improves the representation quality and consequently the performance of each task. This makes the comparison unfair with other methods that utilize only the generalized cross-entropy loss function, including our method. We have conducted a more equitable comparison of these methods in Table 3 (a). When combined with our proposed DAT, Sup+CSI also performs better, although it still falls short compared to when employed on our proposed inter-task variance balancing architecture. When trained with Cross-Entropy loss (also with data augmentation), Sup+CSI performs worse due to the lack of rich representation learned by SupCLR, and it is unable to be combined with DAT.

OOD data: The impact of OOD data is presented in Table 3 (b). The results demonstrate that depending solely on buffer data to differentiate predictions between new and old tasks is inadequate. However, by introducing additional distributionally shifted transformations to augment OOD data, the models trained on previous tasks are somewhat equipped to predict OOD data associated with subsequent tasks. Notably, utilizing both rotation data and buffer data as OOD samples significantly enhances the model's ability to separate classes between tasks.

The trade-off between model size and performance: The results shown in Figure 3 indicate that increasing the compression of the Plastic network (resulting in lower memory consumption) has a minimal impact on the Stable network. This is attributed to the Stable network's ability to gradually acquire knowledge from the Plastic network during the early stages of training via Gradual Knowledge Distillation (GKD). Overall, PS-CLS achieves a favorable trade-off between memory usage and performance.

5 Conclusion

In this paper, we introduced a continual learning approach based on the innovative Plasticity-Stability Complementary Learning system (PS-CLS) framework. PS-CLS addresses persisting challenges in continual learning, particularly the limitations associated with scaling in constrained memory environments and the inter-task class separation problem. By attaining state-of-the-art performance with minimal memory usage, our approach narrows the divide between continual learning and its practical applications in the real world.

References

[1] Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pages 109–165. Elsevier, 1989.

[2] James L McClelland, Bruce L McNaughton, and Randall C O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419, 1995.

[3] Martial Mermillod, Aurélia Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects, 2013.

[4] Gido M Van de Ven and Andreas S Tolias. Three scenarios for continual learning. *arXiv preprint arXiv:1904.07734*, 2019.

[5] Guanxiong Zeng, Yang Chen, Bo Cui, and Shan Yu. Continual learning of context-dependent processing in neural networks. *Nature Machine Intelligence*, 1(8):364–372, 2019.

[6] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2017.

[7] Yaoyao Liu, Yuting Su, An-An Liu, Bernt Schiele, and Qianru Sun. Mnemonics training: Multi-class incremental learning without forgetting. In *Proceedings of the IEEE/CVF conference on Computer Vision and Pattern Recognition*, pages 12245–12254, 2020.

[8] Yue Wu, Yinpeng Chen, Lijuan Wang, Yuancheng Ye, Zicheng Liu, Yandong Guo, and Yun Fu. Large scale incremental learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 374–382, 2019.

[9] Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020.

[10] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017.

[11] Fei Zhu, Xu-Yao Zhang, Chuang Wang, Fei Yin, and Cheng-Lin Liu. Prototype augmentation and self-supervision for incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5871–5880, 2021.

[12] Yu Liu, Sarah Parisot, Gregory Slabaugh, Xu Jia, Ales Leonardis, and Tinne Tuytelaars. More classifiers, less forgetting: A generic multi-classifier paradigm for incremental learning. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXVI 16*, pages 699–716. Springer, 2020.

[13] Sayna Ebrahimi, Mohamed Elhoseiny, Trevor Darrell, and Marcus Rohrbach. Uncertainty-guided continual learning with bayesian neural networks. *arXiv preprint arXiv:1906.02425*, 2019.

[14] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[15] Cuong V Nguyen, Yingzhen Li, Thang D Bui, and Richard E Turner. Variational continual learning. *arXiv preprint arXiv:1710.10628*, 2017.

[16] Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *International conference on machine learning*, pages 4548–4557. PMLR, 2018.

[17] Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. *Advances in Neural Information Processing Systems*, 33:15173–15184, 2020.

[18] Gyuhak Kim, Changnan Xiao, Tatsuya Konishi, Zixuan Ke, and Bing Liu. A theoretical study on solving continual learning. *Advances in Neural Information Processing Systems*, 35:5065–5079, 2022.

[19] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[20] Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*, 2017.

[21] Shipeng Yan, Jiangwei Xie, and Xuming He. Der: Dynamically expandable representation for class incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3014–3023, 2021.

[22] Zichen Miao, Ze Wang, Wei Chen, and Qiang Qiu. Continual learning with filter atom swapping. In *International Conference on Learning Representations*, 2021.

[23] Shiyu Liang, Yixuan Li, and Rayadurgam Srikant. Enhancing the reliability of out-of-distribution image detection in neural networks. *arXiv preprint arXiv:1706.02690*, 2017.

[24] Jihoon Tack, Sangwoo Mo, Jongheon Jeong, and Jinwoo Shin. Csi: Novelty detection via contrastive learning on distributionally shifted instances. *Advances in neural information processing systems*, 33:11839–11852, 2020.

[25] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. pages 3987–3995, 2017.

[26] Hongjoon Ahn, Donggyu Lee, Sungmin Cha, and Taesup Moon. Uncertainty-based continual learning with adaptive regularization. *CoRR*, abs/1905.11614, 2019.

[27] J. Zhang, S. Ghosh, D. Li, S. Tasci, L. Heck, and C. C. J. Kuo. Class-incremental learning via deep model consolidation. In *Proceedings of WACV*, 2020.

[28] Yeming Wen, Dustin Tran, and Jimmy Ba. Batchensemble: an alternative approach to efficient ensemble and lifelong learning. *arXiv preprint arXiv:2002.06715*, 2020.

[29] Quang Pham, Chenghao Liu, and Steven Hoi. Dualnet: Continual learning, fast and slow. *Advances in Neural Information Processing Systems*, 34:16131–16144, 2021.

[30] Prashant Bhat, Bahram Zonooz, and Elahe Arani. Task-aware information routing from common representation space in lifelong learning. *arXiv preprint arXiv:2302.11346*, 2023.

[31] Mustafa Burak Gurbuz and Constantine Dovrolis. Nispa: Neuro-inspired stability-plasticity adaptation for continual learning in sparse networks. *arXiv preprint arXiv:2206.09117*, 2022.

[32] Elahe Arani, Fahad Sarfraz, and Bahram Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. *arXiv preprint arXiv:2201.12604*, 2022.

[33] Dharshan Kumaran, Demis Hassabis, and James L McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in cognitive sciences*, 20(7):512–534, 2016.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[35] Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *International conference on machine learning*, pages 242–252. PMLR, 2019.

[36] Jose M Alvarez and Mathieu Salzmann. Learning the number of neurons in deep networks. *Advances in neural information processing systems*, 29, 2016.

[37] Vivek Ramanujan, Mitchell Wortsman, Aniruddha Kembhavi, Ali Farhadi, and Mohammad Rastegari. What's hidden in a randomly weighted neural network? In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11893–11902, 2020.

[38] Haeyong Kang, Rusty John Lloyd Mina, Sultan Rizky Hikmawan Madjid, Jaehong Yoon, Mark Hasegawa-Johnson, Sung Ju Hwang, and Chang D Yoo. Forget-free continual learning with winning subnetworks. In *International Conference on Machine Learning*, pages 10734–10750. PMLR, 2022.

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

[40] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[41] Hadi Pouransari and Saman Ghili. Tiny imagenet visual recognition challenge. *CS 231N*, 7, 2014.

[42] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.

[43] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: Making all tickets winners. In *International Conference on Machine Learning*, pages 2943–2952. PMLR, 2020.

[44] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. *Advances in neural information processing systems*, 33:18661–18673, 2020.

[45] Fu-Yun Wang, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Foster: Feature boosting and compression for class-incremental learning. In *European conference on computer vision*, pages 398–414. Springer, 2022.

A Variance-constrained sub-networks derivation

Variance-constrained weight Examining a neural network comprising L layers, where the weight matrix and activation function for each layer are denoted as $\mathbf{P}_l^t$ and $g_l(\cdot)$, respectively. Here, l ranges from 1 to L, t ranges from 1 to T, and T represents the total number of learned tasks. The output of the l^{th} layer can be expressed as follows:

$$\mathbf{x}_l^t = \mathbf{P}_l^t g_{l-1}(\mathbf{x}_{l-1}^t) + \mathbf{b}_l^t \tag{8}$$

Here, $\mathbf{x}_0^t = \mathbf{x}^t$ represents the input data for the t^{th} task. To simplify matters, we omit the bias term $\mathbf{b}_l^t$, following the approach taken in the analyses presented in [39, 34]. We posit that the weights associated with each neuron for every task follow to the same distribution represented as w_l^t. The variance of the output for each neuron is then expressed as:

$$\text{Var}[x_{l,n}^t] = K_l \sum_{m=1}^{N_{l-1}^t} \text{Var}[w_{l,n}^t g_{l-1}(x_{l-1,n}^t)] \tag{9}$$

$$= K_l \text{Var}[w_{l,n}^t] \sum_{m=1}^{N_{l-1}^t} \mathbb{E}[g_{l-1}(x_{l-1,m}^t)^2] \tag{10}$$

In accordance with [39], if we assume that $w_{l,n}^t$ follows a symmetric distribution around zero, resulting in $x_{l,n}^t$ having a zero mean and a symmetric distribution around zero, then $\mathbb{E}[g_l(x_{l,n}^t)^2] = \frac{\text{Var}[x_{l,n}^t]}{\alpha_l}$. Here, α_l is determined by the activation function of layer l (e.g., Leaky ReLU: $g_l(x) = \max(0, x) + \beta_l \min(0, x)$, with $\alpha_l = \frac{2}{1+\beta_l^2}$). Plugging this into Equation (10), we get:

$$\text{Var}[x_{l,n}^t] = K_l \text{Var}[w_{l,n}^t] \sum_{m=1}^{N_{l-1}^t} \frac{\text{Var}[x_{l-1,m}^t]}{\alpha_{l-1}} \tag{11}$$

Hence, the sum output variance of each layer changes exponentially depending on the variance of the weights, as follows:

$$\sum_{n=1}^{N_l^t} \frac{\text{Var}[x_{l,n}^t]}{\alpha_l} = \frac{K_l}{\alpha_l} \sum_{n=1}^{N_l^t} \text{Var}[w_{l,n}^t] \sum_{m=1}^{N_{l-1}^t} \frac{\text{Var}[x_{l-1,m}^t]}{\alpha_{l-1}} \tag{12}$$

To prevent the exponential reduction or amplification of output signal magnitudes and to maintain a balance in the magnitudes of output signals across tasks, our objective is to constrain the distribution of the weights as:

$$\frac{K_l}{\alpha_l} \sum_{n=1}^{N_l^t} \text{Var}[w_{l,n}^t] = 1, \quad \mathbb{E}[w_{l,n}^t] = 0, \quad \forall l = \{1, \ldots, L\} \tag{13}$$

This is upheld through the initialization and normalization of the weight distribution. Consequently, the output variance of the final layer become:

$$\sum_{n=N_L^{t-1}}^{N_L^t} \text{Var}[x_{L,n}^t] = \sum_{m=1}^{N_0} \mathbb{E}[(x_{0,m}^t)^2] \tag{14}$$

$$= \sum_{m=1}^{N_0} (\text{Var}[x_{0,m}^t] + (\mathbb{E}[x_{0,m}^t])^2) \tag{15}$$

The output variance of the final layer for each task now solely depends on the input distribution of the t^{th} task, thereby overlooking the impact of training variance on prediction magnitude.

Variance-constrained residual connection: We also account for the variance constraint in the case of residual connections. Let $\mathcal{P}(l)$ denote the set of all layers connected via a residual connection: $x_{l,n}^t = \sum_{p \in \mathcal{P}(l)} g_p(x_{p,n}^t)$. The output variance of layer l can be expressed as:

$$\sum_{p\in\mathcal{P}(l)}\sum_{n=1}^{N_p^t}\frac{\text{Var}[x_{p,n}^t]}{\alpha_p}$$
$$=\sum_{p\in\mathcal{P}(l)}\sum_{n=1}^{N_p^t}\frac{K_p}{\alpha_p}\text{Var}[w_{p,n}^t]\sum_{q\in\mathcal{P}(p)}\sum_{m=1}^{N_q^t}\frac{\text{Var}[x_{q,m}^t]}{\alpha_q}$$

To prevent the reduction or amplification of the magnitudes of the output signals, we impose a constraint:

$$\sum_{p\in\mathcal{P}(l)}\frac{K_p}{\alpha_p}\sum_{n=1}^{N_p^t}\text{Var}[w_{p,n}^t]=1 \tag{16}$$

Variance-constrained Batch Normalization [42]: We adapt the batch normalization to exhibit the behavior described in the analysis above. Specifically, our aim is to ensure:

$$\mathbb{E}[x_{l,n}^t]=0 \tag{17}$$
$$\sum_{n=1}^{N_l^t}\frac{\text{Var}[x_{l,n}^t]}{\alpha_l}=\sum_{m=1}^{N_0}(\text{Var}[x_{0,m}^t]+\mathbb{E}[x_{0,m}^t]^2) \tag{18}$$

To achieve this, we calculate the mean and variance of a mini-batch, denoted as $\mu_l^{(B)}$ and $(\sigma_l^{(B)})^2$, respectively. Subsequently, we normalize the distribution of the mini-batch to align with the intended distribution:

$$x_{l,n}^t=\frac{(x_{l,n}^t-\mu_l^{(B)})\sqrt{\alpha_l\sum_{m=1}^{N_0}(\text{Var}[x_{0,m}^t]+\mathbb{E}[x_{0,m}^t]^2)}}{\sqrt{\sum_{m=1}^{N_l}(\sigma_l^{(B)})^2}} \tag{19}$$

This modification retains the essence of the original Batch Normalization but alters the design distribution from zero mean and unit variance to match the data distribution.

B Experimental settings

We employ the following parameters for training PS-CLS across all scenarios: ResNet18 serves as the backbone with ReLU as the activation layer. The model is trained with a batch size of 32 for 220 epochs. Optimization of the parameters for the Plastic network is done with a learning rate of 0.01, while a learning rate of 0.05 is applied for optimizing the parameters of the stable network. The lambda hyperparameter, controlling memory size, is set to 0.55 for CIFAR-100 with 10 tasks, 1.05 for CIFAR-100 with 20 tasks, 0.11 for Tiny-ImageNet with 5 tasks, and 0.22 for Tiny-ImageNet with 10 tasks. We utilize Erdos-Renyi-Kernel (ERK) [43] sparsity to set mask sparsity for each layer in the stable network, aiming for a total network sparsity of 82%. The coefficient of each component in the general objective function is set to 1. During testing, we generate 32 different views of the original image using the same data augmentation method employed during training.

All experiments are conducted with 5 random seeds and report the mean and standard deviation. We use Pytorch library and conduct experiments on a single A100 GPU.

C Limitations

A component of the proposed method, Variance Balancing, is primarily designed for common feed-forward neural networks like CNNs. Applying it to other architectures, such as Transformers, requires further investigation. Additionally, the augmentation method used for inter-task class separation is currently only described for images. The proposed method achieves a better trade-off between model size and performance, but the number of parameters will eventually increase as the number of tasks grows (discussed in Section I).

D Training Stable networks

The Stable networks are trained using the Edge Popup algorithm [37], similar to SupSup [17]. Specifically, given the knowledge base of the current task t, denoted as $\mathbf{P}_l^{t-1}\cup\mathbf{R}_l^t$, we seek binary masks $\mathbf{M}_l^t$ for task t that minimize the cross-entropy loss. These

Loss	Method	CIFAR100/10T		CIFAR100/20T		TinyImg/5T		TinyImg/10T	
		TIL(%)↑	CIL(%)↑	TIL(%)↑	CIL(%)↑	TIL(%)↑	CIL(%)↑	TIL(%)↑	CIL(%)↑
SupCLR [44]	*Sup+CSI*	93.0	66.6	95.3	60.5	65.9	47.7	74.1	46.3
Cross-entropy	*Sup+CSI**	90.1	60.2	93.2	53.2	63.9	41.3	75.7	45.6
	*Sup+CSI** w/ DAT	90.1	58.1	93.2	49.5	63.9	42.2	75.7	43.6
	PS-CLS w/o DAT	89.6	63.9	92.9	60.8	68.7	48.0	74.6	45.2
	PS-CLS	93.0	71.1	95.4	67.5	73.2	53.5	79.0	51.4

Table 4: Compare with Sup+CSI [18] when employing cross-entropy loss and implementing data augmentation during testing. "Sup+CSI*" denotes the version in which we re-implement Sup+CSI with cross-entropy loss. "DAT" denotes our proposed method of using data augmentation during training and testing

binary masks $\mathbf{M}_l^t$ are derived by selecting the top $p\%$ of entries in the score matrices $\mathbf{V}_l$, with p determining the sparsity of the mask $\mathbf{M}_l^t$. We optimize $\mathbf{V}_l$ using straight-through gradients during training, and subsequently discard $\mathbf{V}_l$ after obtaining the final binary masks $\mathbf{M}_l^t$ for task t.

E Compute number of parameters (NP)

The total number of parameters in the PS-CLS model after learning all T tasks comprises the parameters in the final plastic network $\mathbf{P}^T$ and the parameters of the binary masks $\mathbf{M}^t$ for the Stable networks of each task. Following the methodology outlined in [38], we compress the binary masks $\mathbf{M}^t$ for the Stable networks to conserve memory. Initially, for compact compression, we transform a series of binary masks into a single cumulative decimal mask and encode each integer as an ASCII code to represent a distinct symbol. Subsequently, using these symbols, we apply a lossless compression algorithm—specifically, Huffman encoding (Huffman, 1952). This encoding method achieved approximately 78% compression rate for 7-bit binary masks and allowed for decompression without loss of bits. Moreover, we observed that the compression rate increases sublinearly with the size of binary bits. To calculate the number of parameters in PS-CLS, we use the formula: $NP = |\mathbf{P}^T| + T\frac{1-\alpha}{32}S$, where the average compression rate α is approximately 0.78, determined through 7-bit Huffman encoding, and S represents the number of parameters in the backbone network (ResNet18 base with 11.2 million parameters in the experiments). The average compression rate α varies depending on the size of bit-binary maps and the compression method employed. We assume all task weights of 32-bit precision.

F A fair comparison with [18]

Sup+CSI and HAT+CSI [18] necessitate an extra representation learning phase involving supervised contrastive loss (SupCLR) [44]. This feature enables the method to acquire a significantly enhanced representation of the data, introducing an unfairness when compared to other baselines that lack representation learning phases during training—our method being one of them. To ensure a more equitable comparison with Sup+CSI [18], we eliminate the SupCLR loss component during the training of CSI [24]. In its place, we incorporate the standard cross-entropy loss used in training our method and other baselines. It's important to note that this adjustment does not alter the fundamental concept of CSI [24], which involves employing distribution-shifted transforms both in training and testing to enhance the model's out-of-distribution (OOD) detection capability. We retain the methodology of utilizing data augmentation during training, consistent with our approach, and maintain the same set of hyperparameters for the Sup+CSI method. Furthermore, by employing the identical data augmentation technique during training, we assess the efficacy of our approach, which involves using data augmentation during testing, when applied to Sup+CSI. The reported results in Table 4 reveal a substantial decrease in the performance of Sup+CSI* in both TIL and CIL settings, when the representation learning phase with SupCLR loss is omitted. Particularly in the CIL setting, where the most significant decrease occurs, it is evident that the absence of SupCLR substantially impacts the model's out-of-distribution (OOD) detection ability. Combining Sup+CSI* with our data augmentation during testing technique (DAT) maintains unchanged results for TIL, but the outcomes for CIL deteriorate. This indicates that, despite utilizing the same ensemble of transformed versions of the original data (in this case, distributionally shifted transforms), the out-of-distribution (OOD) data prediction strategy of Sup+CSI is not compatible when integrated with our proposed method. Conversely, when coupled with our suggested architecture, DAT proves highly effective, significantly enhancing results for both TIL and CIL. In summary, each component of our proposed method, PS-CLS, plays a crucial role in mutually reinforcing its effectiveness for the intended purpose.

G Evaluation time:

We further investigate the evaluation time and variance in Table 5. [18] employs a 90-degree rotation at inference, resulting in a higher time cost compared to our base method. PS-CLS, leveraging data augmentation at inference, can be run in parallel, slightly increasing the inference time while significantly enhancing performance. With faster inference times (N<16), PS-CLS clearly outperforms [18]. The variance in the inference of PS-CLS is not high and is further reduced when increasing the number of views (N).

Table 5: Evaluation on CIFAR-100/20 with varying extra views (N) produced by data augmentation per test sample, using a single A100 GPU. "Sup+CSI" denotes evaluation with the pre-trained model from [18]. "Sup+CSI*" indicates evaluation with 32 extra views per test sample. "Time" represents evaluation time for both CIL and TIL in seconds. Results reflect mean and standard deviation across 8 different random seeds.

PS-CLS	N=0	N=2	N=4	N=8	N=16	N=32	N=48	Sup+CSI	Sup+CSI*
CIL (%)	60.8±0.00	62.5±0.28	64.3±0.23	65.5±0.40	66.6±0.20	67.1±0.12	67.3±0.13	59.3±0.00	63.2±0.17
TIL (%)	92.9±0.00	94.1±0.09	94.7±0.09	95.0±0.12	95.3±0.07	95.4±0.07	95.5±0.05	95.0±0.00	97.0±0.13
Time	234s	270s	322s	480s	781s	1379s	1994s	632s	2074s

Method	PS-CLS	DER [21]	FOSTER [45]
AVG CIL (%) ↑	75.60	67.98	70.65
NP (Million) ↓	017.3	224.0	011.2

Table 6: The average incremental accuracy (AVG CIL) on CIFAR-100 with 20 tasks is compared between PS-CLS and the state-of-the-art expansion-based methods DER [21] and FOSTER [45].

H Compare with expansion-based method

In Table 6, we present the average incremental accuracy (AVG CIL) of PS-CLS on CIFAR-100 with 20 tasks and compare it with state-of-the-art expansion-based methods DER [21] and FOSTER [45]. It's worth noting that our method employs ResNet18 as a backbone, whereas [21, 45] use ResNet32. Despite this difference, PS-CLS consistently outperforms the expansion-based baselines, demonstrating superior performance with only a minimal increase in parameters.

I Long sequence of tasks

Figure 4 shows the performance on the Tiny ImageNet dataset split into 40 tasks. Even after learning 40 tasks, our method still achieves a high Class Incremental Learning (CIL) performance of 44.24%. In comparison, the performance when the dataset is split into 10 tasks and 5 tasks is 51.4% and 53.5%, respectively. This demonstrates the efficiency of our proposed method in handling long sequences of tasks. Moreover, the memory utilization of our proposed method remains robust, with only a linear increase, while the overall number of parameters remains smaller than the number of parameters of the backbone used by baselines, which is 40 million parameters.

J Analysis of the structure of Stable Networks

Figure 5 illustrates how the network utilizes old weights in the knowledge base at each layer. It's evident that the higher layers, closer to the input, mostly reuse old weights. This is because the model learns more general features at these layers, which can be effectively reused. Conversely, the lower layers are responsible for learning task-specific features, as indicated by the preference for selecting more randomly initialized weights (unknown knowledge). Additionally, we observe that the model tends to reuse more old weights as the number of tasks increases, indicating that the knowledge base becomes richer and more useful for handling new tasks.

Figure 6 provides further insight into how weights from each old task and random weights are utilized. We also observe that the number of random weights used decreases as the number of tasks increases.

K Proximal Gradient descent and training algorithm

We employ proximal gradient descent to optimize the sparsity regularization and prune the network during training. After updating the Plastic weights with SGD, we then perform the proximal operator as follows:

$$\mathbf{P}^t_{l,n,:} = \left(1 - \frac{\gamma\lambda_l}{\sqrt{\text{Var}[w^t_{l,n}]}}\right)_+ P^t_{l,n,:} \tag{20}$$

Here, γ represents the learning rate, and λ_l is the hyperparameter that governs the plasticity of the model. Algorithm 1 outlines the detailed complementary learning process for each task's Plastic and Stable sub-networks.

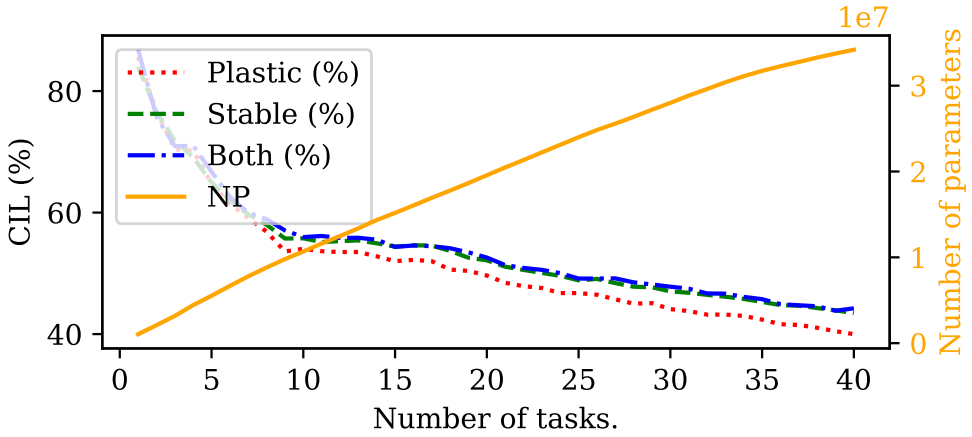


Figure 4: CIL performance and number of parameters usage for each task in TinyImg/40T

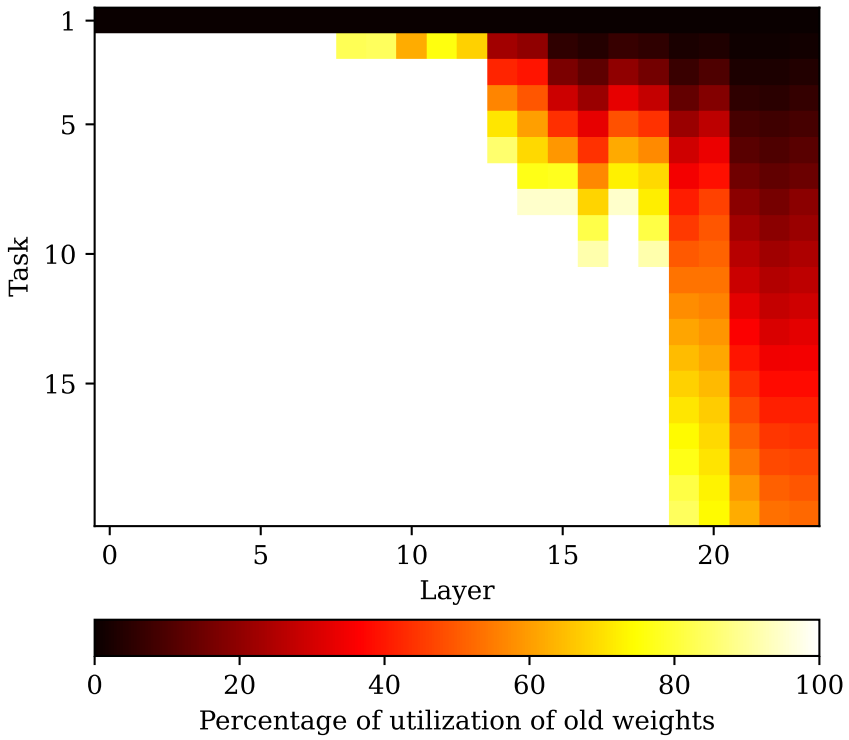


Figure 5: The utilization percentage of old weights at each layer. A smaller layer index corresponds to layers closer to the input. The weights are used when they are selected by the binary mask of the Stable network.

Algorithm 1 Learning *Plastic* and *Stable* networks

for $t = 1, 2, \ldots T$ **do**
  Over-parameterized expand
  while not converged **do**
    Compute Loss
    Update model's parameters with SGD
    Perform proximal operator
    Normalize weights
  end while
end for

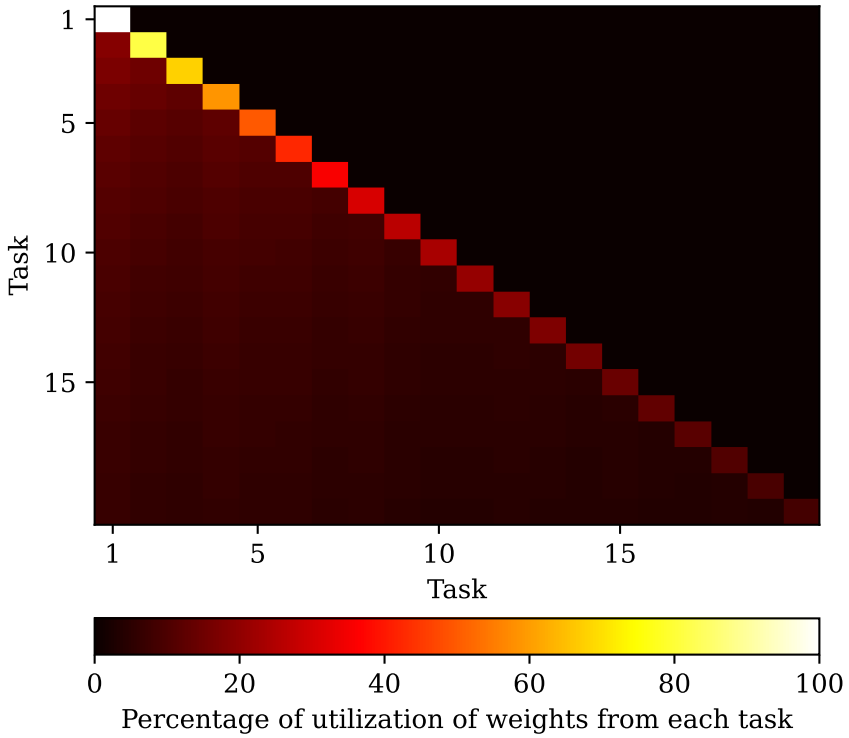


Figure 6: The percentage of weights utilized from each task. The weights are used when they are selected by the binary mask of the Stable network. The entries along the diagonal represent the randomly initialized weights. We can observe how each task utilizes the weights from the knowledge base of previous tasks and the randomly initialized weights.